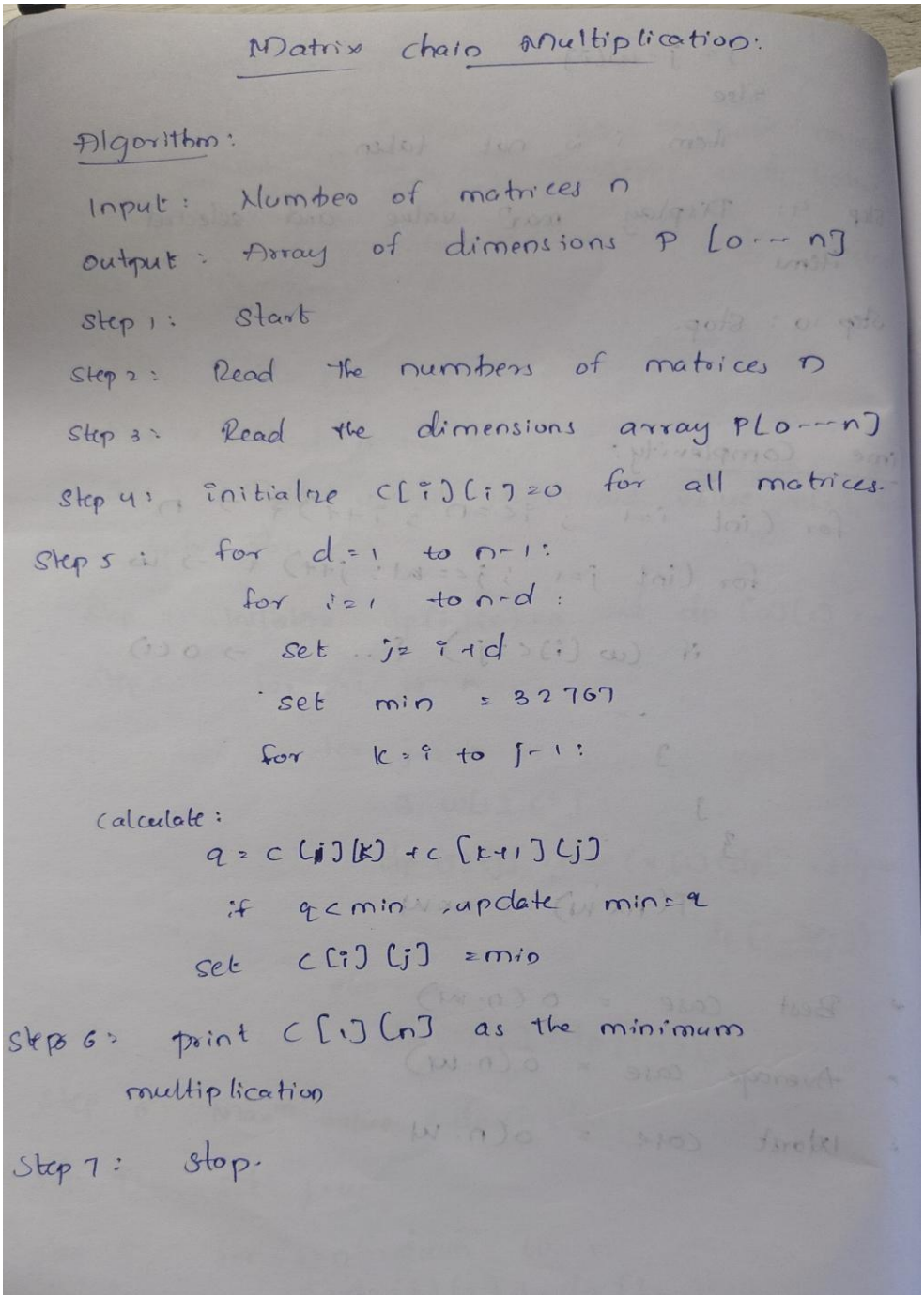
PRACTICAL: 06

Aim: Implementation of chain matrix multiplication of using dynamic programming.

Algorithm:



Matrix chain Multiplication:

Algorithm:

Input: Number of matrices n

output: Array of dimensions P [0 --- n]

Step 1: Start

Step 2: Read the numbers of matrices n

Step 3: Read the dimensions array P[0---n]

Step 4: initialize c[i][i]=0 for all matrices.

Step 5: for d=1 to n-1:

for i=1 to n-d:

set j= i+d

set min = 32767

for k=i to j-1:

calculate:

q = c[i][k] + c[k+1][j]

if q<min, update min=q

set c[i][j] =min

Step 6: print c[1][n] as the minimum multiplication

Step 7: stop.

Program / Code:

```
#include  <iostream>
using namespace std;

class MatrixChain{
private:
   int p[20];
   int c[20][20];
   int n;

public:
   void getInput()
   {
      cout << "Enter number of matrices: ";
      cin >> n;

      cout << "Enter dimensions: ";
      for (int i = 0; i <= n; i++)
      {
         cin >> p[i];
      }
   }

   void matrixChain()
   {
      // Cost of multiplying one matrix is 0
      for (int i = 1; i <= n; i++)
```

```
{
   c[i][i] = 0;
}

// d = chain length - 1
for (int d = 1; d < n; d++)
{
   for (int i = 1; i <= n - d; i++)
   {
      int j = i + d;
      int min = 32767;

      for (int k = i; k <= j - 1; k++)
      {
         int q = c[i][k]
               + c[k + 1][j]
               + p[i - 1] * p[k] * p[j];

         if (q < min)
         {
            min = q;
         }
      }

      c[i][j] = min;
   }
}
```

```
        cout << "Minimum number of multiplications: "
             << c[1][n] << endl;
    }
};

int main()
{
    MatrixChain obj;

    obj.getInput();
    obj.matrixChain();

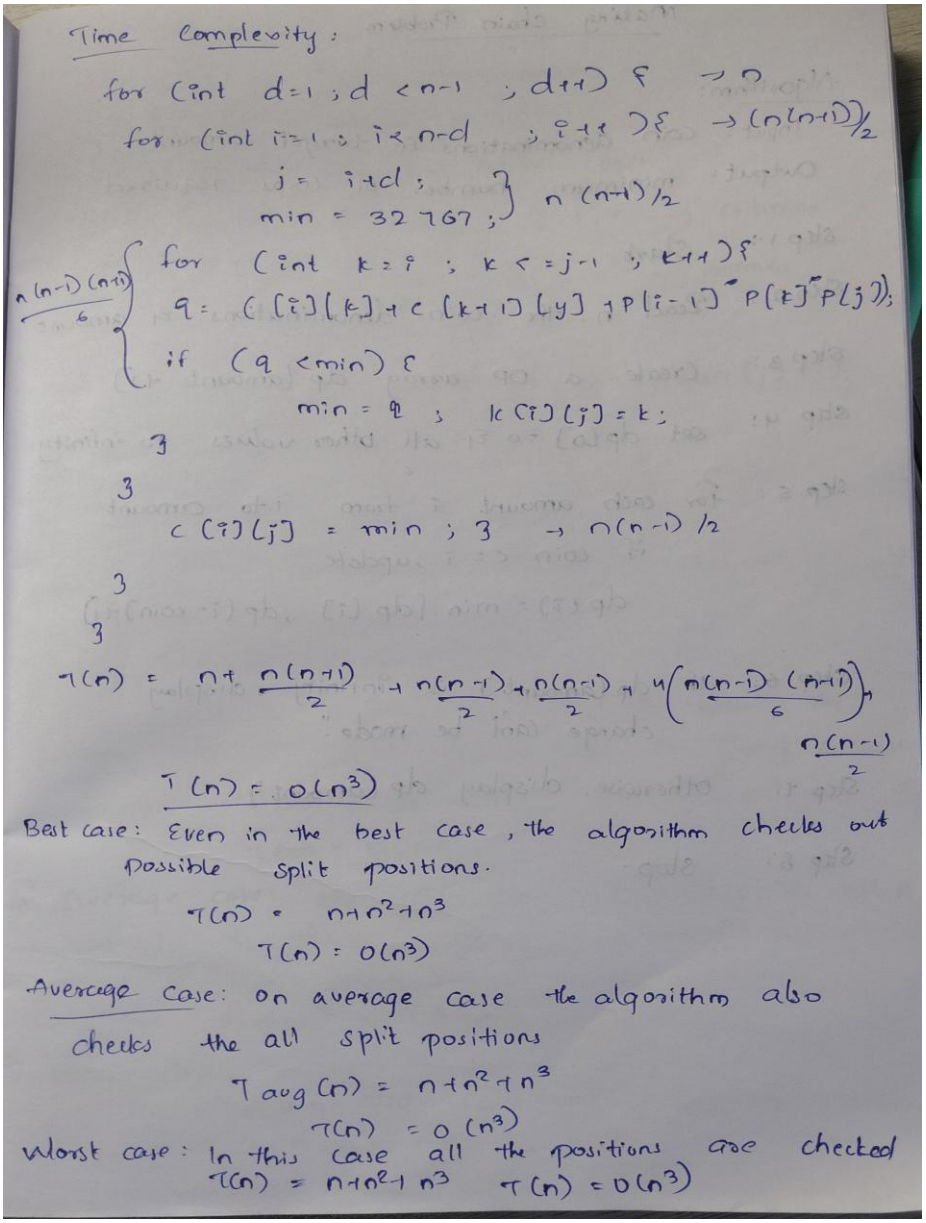
    return 0;
}
```

Output:

```
PS C:\Users\pusap\Downloads\c++> C:\Users\pusap\Downloads\c++\matrixchainmultipication.exe
Enter number of matrices: 5
Enter dimensions: 4

2
3
4
5
6
Minimum number of multiplications: 172
```

Time Complexity:



Time Complexity :

```
for (int d=1 ; d <n-1 ; d++) {                    → n
    for (int i=1 ; i<= n-d ; i++ ){               → (n(n+1))/2
        j = i+d ;                                 } n(n+1)/2
        min = 32767 ;
        for (int k = i ; k <= j-1 ; k++){         n(n-1)(n+1)/6
            q = c[i][k] + c[k+1][y] + p[i-1] * p[k] * p[j];
            if (q <min) {
                min = q ; k[i][j] = k ;
            }
        }
        c[i][j] = min ; }                         → n(n-1)/2
    }
}
```

$T(n) = n + \frac{n(n+1)}{2} + \frac{n(n-1)}{2} + \frac{n(n-1)}{2} + 4\left(\frac{n(n-1)(n+1)}{6}\right)$, $\frac{n(n-1)}{2}$

$T(n) = O(n^3)$

Best case: Even in the best case, the algorithm checks out possible split positions.

$T(n) = n + n^2 + n^3$

$T(n) = O(n^3)$

Average case: on average case the algorithm also checks the all split positions

$T_{avg}(n) = n + n^2 + n^3$

$T(n) = O(n^3)$

Worst case: In this case all the positions are checked

$T(n) = n + n^2 + n^3$ $T(n) = O(n^3)$

Conclusion:

The Matrix Chain Multiplication problem was successfully implemented using Dynamic Programming. The algorithm determines the optimal order of multiplying matrices while minimizing the total number of scalar multiplications. By storing the results of smaller subproblems in a DP table, the overall problem can be solved efficiently and systematically.